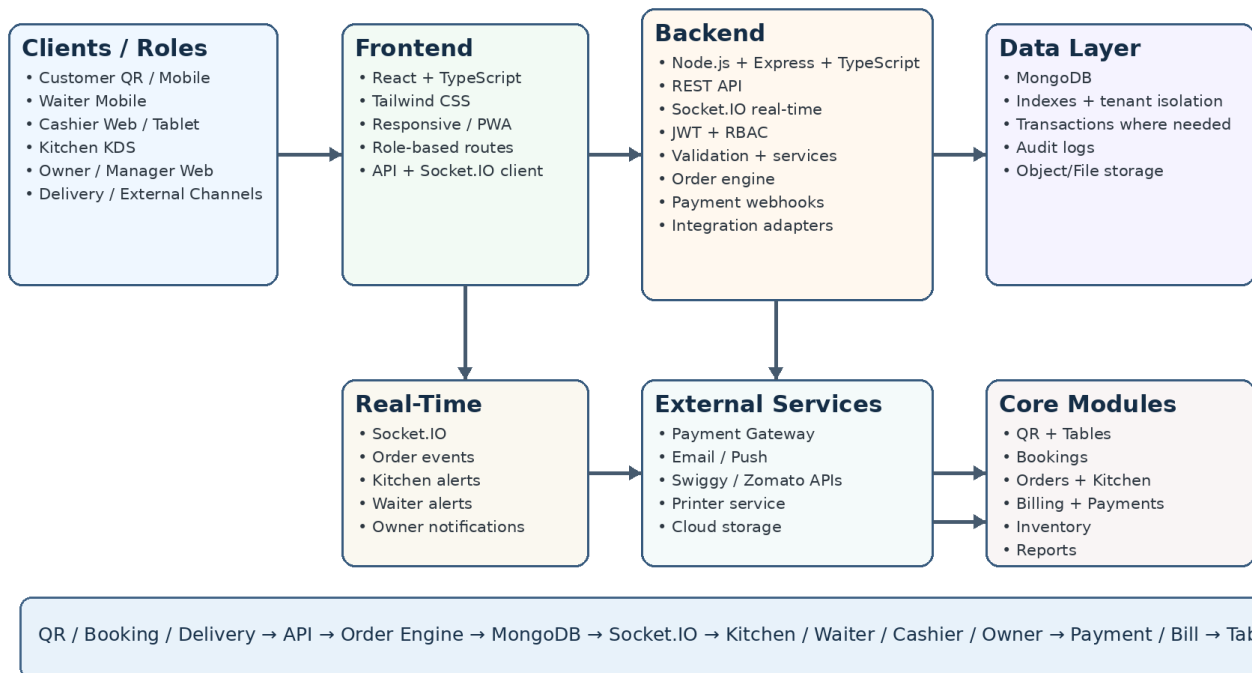


RESTAURANT MANAGEMENT PLATFORM

Codex-Ready Master Product, UI/UX & Engineering Specification

One platform for QR ordering, table booking, waiter operations, kitchen display, cashier/POS, payments, billing, inventory, notifications, analytics and online delivery integrations.

Restaurant Management Platform — System Architecture



Primary implementation target: React + TypeScript frontend, Node.js + Express + TypeScript backend, MongoDB, Socket.IO, REST APIs, RBAC, secure payment webhooks, and integration adapters.

1. Product Goal

Create a production-ready multi-restaurant SaaS platform. A restaurant owner creates a restaurant, menu, tables and staff. Every table automatically receives a QR code. Customers scan it, order, track the order and pay. The same backend sends orders to kitchen, waiter, cashier and owner in real time.

2. Core Roles

Role	Main responsibility
Owner	Full restaurant control, settings, menu, staff, reports, payments, integrations.
Manager	Daily operations, tables, bookings, staff assignments, reports.
Cashier	Bills, payments, printing, order/payment verification.
Kitchen/Chef	Live kitchen orders, preparation and ready status.
Waiter/Server	Assigned tables, seating, bookings, manual orders, customer requests, bills, serving.
Customer	QR menu, ordering, booking, bill, payment, order tracking.

3. Customer QR + Table System

- Owner creates a table with number, capacity, floor/section and optional description.
- Backend creates a unique public QR identifier/URL for that table.
- QR opens the restaurant menu and starts/joins a table session.
- Never put passwords, private secrets or payment credentials inside the QR.
- Multiple customers can use the same table session; support combined or split bills.
- Table states: AVAILABLE, RESERVED, OCCUPIED, ORDERING, FOOD_READY, BILL_REQUESTED, PAYMENT_PENDING, CLEANING.

```
GET /public/restaurants/:restaurantId/tables/:tableToken/menu
POST /public/table-sessions
POST /orders
GET /orders/:orderId
```

4. Digital Menu & Ordering

- Categories, products, images, descriptions, prices, tax, add-ons, availability and optional recipes.
- Customer cart, quantity changes, special instructions and order confirmation.
- Backend recalculates all prices from the database; never trust totals from the browser.
- Order source is stored: DINE_IN_QR, DINE_IN_WAITER, CASHIER, SWIGGY, ZOMATO, WEBSITE, PHONE or OTHER.
- Customer sees order state in real time.

5. Waiter Dashboard — Dedicated Mobile UI

- Live table map with AVAILABLE, RESERVED, OCCUPIED, READY, BILL REQUESTED and CLEANING states.
- My assigned tables, current customers, seating time, guest count and active order.
- Upcoming bookings and pre-booked tables with ARRIVED / SEATED / NO_SHOW actions.
- Manual order entry when a customer orders directly with the waiter.
- Add items to an existing order without losing the original order history.
- Food-ready notifications from kitchen.
- Customer requests: call waiter, water, extra plate, cutlery, napkin, other request.
- Bill request and authorized cash collection.

- Table closing → cleaning → available.
- Optional table transfer between waiters with an audit record.

6. Booking System

- Customers or staff can reserve a table by date, time, guest count and optional notes.
- Booking states: PENDING, CONFIRMED, ARRIVED, SEATED, COMPLETED, CANCELLED, NO_SHOW.
- Prevent double booking using backend availability checks and database transaction/constraint logic.
- Waiter receives configurable booking reminders.

7. Kitchen Display System (KDS)

- Real-time queue for dine-in, takeaway and delivery orders.
- Order card contains order number, source, table/external order ID, items, modifiers, notes and elapsed time.
- Actions: NEW → ACCEPTED → PREPARING → READY.
- Waiter receives READY event for dine-in orders.
- Delivery orders show platform/external order ID rather than a restaurant table.

8. Cashier / POS / Billing

- Active tables, orders, bill requests, payment status and recent bills.
- Generate bill from server-side order data.
- Support cash, UPI/card/payment gateway methods as configured.
- Cash payment records amount received, change, collector and timestamp.
- Print thermal/A4 bill and generate PDF/digital bill.
- Bill QR can open a safe digital bill page.
- Refunds and payment corrections require permission and create audit logs.

9. Online Delivery

Create an integration layer so supported providers such as Swiggy and Zomato can feed orders into the same order engine using their official partner/API mechanisms where available. Do not scrape or automate consumer websites in violation of provider terms.

- Adapter pattern: SwiggyAdapter, ZomatoAdapter, WebsiteOrderAdapter, FutureProviderAdapter.
- Map external orders into one internal order model.
- Store provider, externalOrderId, payment state and delivery status.
- Kitchen sees delivery orders alongside dine-in orders with a source badge.
- Cashier/owner can filter sales by channel.

10. Inventory

- Stock items, units, suppliers/receipts if added later, low-stock thresholds and inventory transactions.
- Optional recipe/BOM per menu product.
- When an order is completed, deduct stock according to the configured recipe.
- Never silently alter stock: every adjustment creates an inventory transaction.

11. Notifications

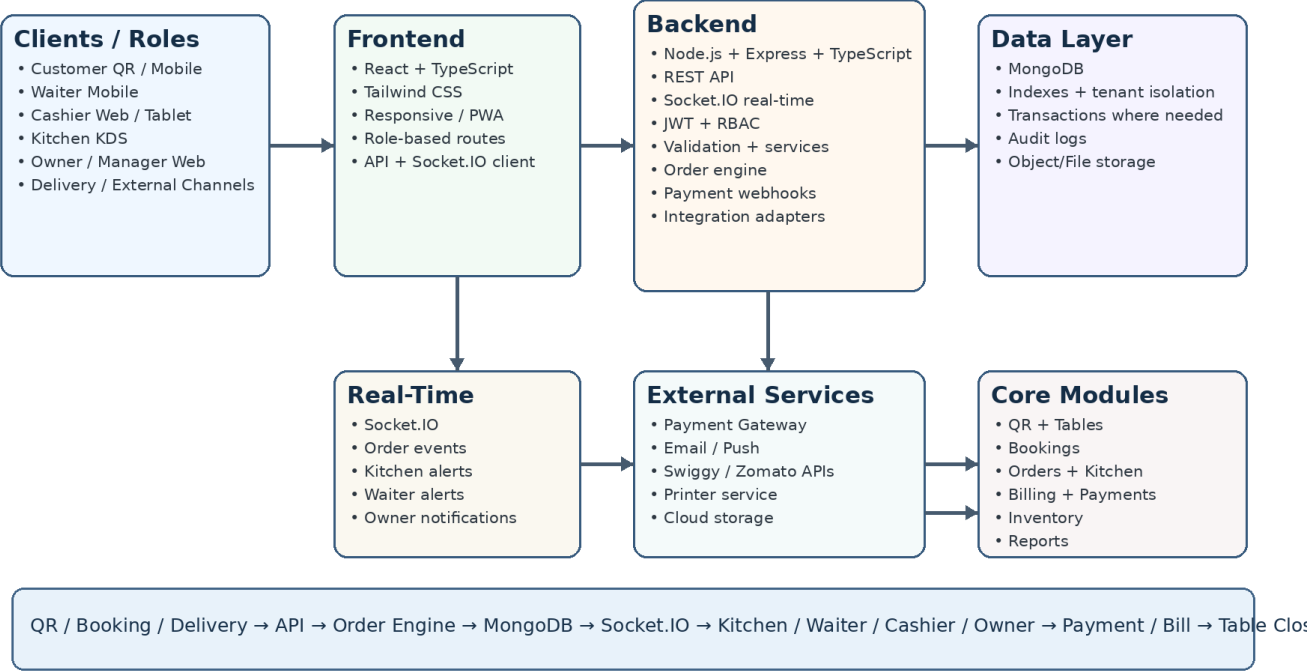
- Socket.IO in-app events for new order, food ready, bill requested, customer request, payment success, booking reminder and low stock.
- Optional web push/mobile push, email or SMS through provider services.
- Notification collection stores recipient, type, entity, read state, createdAt and delivery metadata.

12. Owner / Manager Analytics

- Sales by day/week/month/year.
- Dine-in vs delivery vs takeaway.
- Cash vs online payments.
- Order volume, average order value, top products, table occupancy, peak hours.
- Tax/payment summaries and cashier/waiter operational reports.
- Exportable reports can be added after core functionality is stable.

13. Technical Architecture

Restaurant Management Platform — System Architecture



- Frontend: React + TypeScript + Tailwind CSS. Use responsive layouts; waiter/customer interfaces are mobile-first.
- Backend: Node.js + Express + TypeScript. Separate controllers, services, repositories/models, middleware and integrations.
- Database: MongoDB with restaurantId on tenant-owned records and carefully designed indexes.
- Real-time: Socket.IO rooms such as restaurant:{id}, kitchen:{id}, waiter:{userId}, table:{id}.
- Authentication: short-lived access token + secure refresh-token strategy. Passwords hashed with a strong password hashing library.
- Validation: Zod or equivalent schema validation at API boundaries.
- Object storage: restaurant/product images, QR images and generated documents.
- Payments: provider abstraction + signed webhook verification; never trust frontend payment success.

14. Recommended Backend Folder Structure

```
backend/  
src/  
app.ts  
server.ts  
config/  
db/  
middleware/  
auth.middleware.ts  
rbac.middleware.ts  
validation.middleware.ts  
error.middleware.ts  
modules/  
auth/  
restaurants/  
users/  
staff/  
roles/  
tables/  
qr/  
bookings/
```

```
menu/  
orders/  
kitchen/  
waiters/  
billing/  
payments/  
inventory/  
notifications/  
reports/  
integrations/  
swiggy/  
zomato/  
sockets/  
utils/  
types/  
tests/
```

15. Recommended Frontend Structure

```
frontend/  
src/  
app/  
routes/  
layouts/  
components/  
ui/  
tables/  
orders/  
menu/  
billing/  
notifications/  
features/  
auth/  
owner/  
manager/  
cashier/  
kitchen/  
waiter/  
customer/  
bookings/  
delivery/  
inventory/  
services/  
api.ts  
socket.ts  
hooks/  
stores/  
types/  
utils/
```

16. Core Database Models

- User: `_id`, `restaurantId`, `roleId`, `name`, `phone/email`, `passwordHash`, `status`, `timestamps`
- Restaurant: `ownerId`, `name`, `address`, `tax settings`, `hours`, `branding`, `settings`
- Table: `restaurantId`, `number`, `capacity`, `section`, `status`, `qrToken`
- TableSession: `restaurantId`, `tableId`, `openedAt`, `closedAt`, `guest count`, `status`
- Booking: `restaurantId`, `tableId`, `customer info`, `date/time`, `guests`, `status`
- Category / Product / Addon: `restaurantId`, `menu data`, `pricing`, `availability`
- Order: `restaurantId`, `tableId?`, `customerId?`, `waiterId?`, `source`, `items`, `totals`, `status`, `paymentStatus`
- KitchenOrder: `orderId`, `station`, `status`, `timestamps`
- Bill: `orderId`, `billNumber`, `totals`, `tax`, `payment state`, `print state`
- Payment: `restaurantId`, `orderId`, `method`, `amount`, `provider`, `transactionId`, `status`, `verifiedAt`

- Notification: recipientId, restaurantId, type, entityId, title, readAt
- InventoryItem / Recipe / InventoryTransaction
- AuditLog: restaurantId, actorId, action, entityType, entityId, metadata, timestamp

17. Core API Groups

Area	Example endpoints
Auth	POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout
Tables/QR	GET/POST/PATCH /tables, POST /tables/:id/qr, GET /public/menu/:token
Bookings	GET/POST/PATCH /bookings, POST /bookings/:id/check-in
Menu	GET/POST/PATCH/DELETE /categories, /products, /addons
Orders	POST /orders, GET /orders, PATCH /orders/:id/status
Kitchen	GET /kitchen/orders, PATCH /kitchen/orders/:id/status
Billing	POST /bills, GET /bills/:id, POST /bills/:id/print
Payments	POST /payments, POST /payments/webhook/:provider
Inventory	GET/POST/PATCH /inventory, POST /inventory/adjustments
Notifications	GET /notifications, PATCH /notifications/:id/read
Reports	GET /reports/sales, /reports/payments, /reports/products

18. Real-Time Event Design

```

order.created
order.accepted
order.preparing
order.ready
order.served
order.cancelled

booking.created
booking.confirmed
booking.arrived
booking.no_show

customer.requested
bill.requested
payment.succeeded
payment.failed
table.cleaned
table.available
inventory.low_stock

```

Use authorization checks before emitting sensitive events. Rooms must be scoped by restaurant and, where appropriate, role/user/table.

19. Security & Multi-Tenant Isolation

- Every restaurant-owned record includes restaurantId.
- Every authenticated request derives restaurant scope from the authenticated user, not from a client-supplied restaurantId alone.
- Use RBAC on the server for every protected action.
- Validate body/query/params; rate-limit public QR endpoints and login.
- Use secure headers, strict CORS, secure cookies/tokens as appropriate, and secret values only in environment/secret management.
- Verify payment webhooks cryptographically.

- Audit sensitive actions: price changes, refunds, cancellations, cash payments, role changes, table transfers and inventory adjustments.
- Do not log passwords, payment secrets or sensitive tokens.

20. Codex Implementation Rules

Give Codex small, verifiable implementation tasks rather than asking it to generate the entire application in one pass. After every module, run type checking, linting, tests and a focused manual flow.

- Keep TypeScript strict mode enabled.
- Create shared types/schemas for API request/response contracts.
- Do not duplicate business rules in React and Node.js.
- Put business logic in backend services; controllers should stay thin.
- Use reusable UI components and role-specific layouts.
- Use controlled database initialization/migration scripts for database changes.
- Add seed data only for development; never use hardcoded production users/orders.
- Every new feature must include authorization, validation, error handling and tests.
- Never mark payment successful from the frontend without verified server-side confirmation.
- Never let the client choose an arbitrary restaurantId to access data.

21. Codex Build Sequence

Phase 1	Repository + tooling	Monorepo or clearly separated frontend/backend, TypeScript, environment configuration, lint, format, test runner, CI.
Phase 2	Auth + RBAC	User accounts, roles, permissions, restaurant tenancy, login/refresh/logout.
Phase 3	Restaurant setup	Restaurant profile, menu categories/products, tables, QR generation.
Phase 4	Customer QR	Public QR menu, table session, cart, order creation and tracking.
Phase 5	Kitchen	KDS, real-time order events, preparation workflow.
Phase 6	Waiter	Mobile dashboard, table map, bookings, manual orders, requests, ready alerts, bill requests.
Phase 7	Cashier	POS, bill calculation, payment records, print/PDF.
Phase 8	Bookings	Availability, reservation lifecycle, reminders and conflict prevention.
Phase 9	Inventory	Items, recipes, deductions, adjustments, low-stock alerts.
Phase 10	Delivery integrations	Official provider integrations/adapters and unified delivery order workflow.
Phase 11	Analytics	Sales, payment, product, occupancy and operational reports.
Phase 12	Hardening	Security tests, concurrency tests, payment webhook tests, backup/recovery, production deployment.

22. UI Screen Inventory — Build Each as a Separate Screen

- 01 Login / Role Selection
- 02 Owner Dashboard
- 03 Manager Dashboard
- 04 Cashier Dashboard
- 05 Kitchen Display / KDS
- 06 Waiter Mobile Dashboard
- 07 Waiter Table Details
- 08 Waiter Live Table Map
- 09 Waiter Bookings
- 10 Waiter Manual Order
- 11 Waiter Customer Requests
- 12 Waiter Bill & Payment
- 13 Waiter Notifications
- 14 Customer QR Menu
- 15 Customer Cart
- 16 Customer Order Tracking
- 17 Customer Table Booking
- 18 Customer Digital Bill
- 19 Online Delivery Orders
- 20 Delivery Order Details
- 21 Menu Management
- 22 Product Editor
- 23 Table Management + QR Generator
- 24 Booking Management
- 25 Staff & Role Management
- 26 Payment Management
- 27 Bill/Invoice Management
- 28 Inventory Dashboard
- 29 Inventory Item / Recipe
- 30 Notifications Center
- 31 Reports & Analytics
- 32 Restaurant Settings
- 33 Integration Settings
- 34 Audit Logs
- 35 Mobile Waiter Profile

23. UI Design Rules for Codex

- Use a consistent design system: spacing, typography, buttons, cards, tables, badges, modals and form controls.
- Desktop owner/cashier/manager screens should use a persistent sidebar and responsive content.

- Kitchen screen should prioritize large order cards, elapsed time, status actions and high readability.
- Waiter UI should be mobile-first with bottom navigation and large touch targets.
- Customer QR menu should be fast, minimal and optimized for mobile browsers; avoid requiring an app install.
- Every asynchronous action needs loading, success, empty and error states.
- Every table/order/payment status must have a clear visual badge and text label, not color alone.
- Do not make important financial actions one-click without confirmation.

24. End-to-End System Flow

```

CUSTOMER
↓ scan QR / book table
PUBLIC MENU
↓ cart
ORDER API
↓
ORDER ENGINE
├─ MongoDB
├─ Socket.IO → Kitchen
├─ Socket.IO → Waiter
├─ Socket.IO → Cashier
└─ Socket.IO → Owner

KITCHEN
NEW → ACCEPTED → PREPARING → READY
↓
WAITER
↓
SERVED
↓
BILL REQUEST
↓
CASHIER
↓
CASH / ONLINE PAYMENT
↓
VERIFIED
↓
BILL CLOSED
↓
CLEANING → AVAILABLE

DELIVERY:
SWIGGY/ZOMATO → Integration Adapter → Order Engine → Kitchen → Ready → Pickup → Completed

```

25. Testing Checklist

- Unit tests for pricing, tax, discounts, table states, booking conflicts and order transitions.
- Integration tests for authentication, RBAC, orders, payments and webhooks.
- Real-time tests for kitchen/waiter/cashier event delivery.
- Concurrency tests: two customers ordering from the same table; two staff members updating the same order; two users trying to reserve the same table.
- Security tests for cross-restaurant access and privilege escalation.
- Payment tests for success, failure, timeout, duplicate webhook and refund.
- Printer/PDF tests and mobile responsive testing.
- End-to-end test: book → arrive → QR order → kitchen → serve → bill → pay → clean → available.

26. Production Checklist

- Environment variables and secrets are configured outside source control.

- Production database backups and restore procedure tested.
- HTTPS everywhere.
- Monitoring and structured server logs.
- Error tracking.
- Rate limiting and abuse protection.
- Database indexes verified.
- Payment webhook endpoint protected and verified.
- Delivery integrations tested in their approved sandbox/production process.
- Role permissions reviewed before launch.

27. The Codex Master Instruction

Use this document as the source of truth. Implement the platform incrementally. Before coding each feature, inspect the existing repository and preserve working functionality. Do not replace working modules unnecessarily. For every feature, first create the data model/API contract, then backend service/controller/tests, then frontend UI, then real-time events, then integration tests. Do not use fake data except explicit development seed data. Keep the application production-oriented, secure, multi-tenant and type-safe.

28. Suggested First Codex Prompt

Read the Restaurant Management Platform specification PDF as the source of truth.

First inspect the existing repository and report:

1. Current frontend framework and version
2. Current backend framework and version
3. Database and existing schemas
4. Authentication implementation
5. Existing folder structure
6. Existing reusable components
7. Existing API patterns
8. Existing tests
9. Deployment configuration

Do NOT rewrite the project.

Then create an implementation plan for Phase 1 only:

- repository/tooling
- environment configuration
- strict TypeScript
- linting/formatting
- testing
- backend foundation
- frontend foundation
- shared types

Implement only Phase 1 after the plan is validated.

Run typecheck, lint and tests.

Report changed files and any remaining issues.

29. Important Implementation Principle

Do not ask Codex to generate hundreds of thousands of lines at once. A reliable production system is built as small modules with contracts and tests. This specification is intentionally structured so Codex can implement one phase/module at a time while preserving the architecture.